CHAPTER 3

Vehicle OS and Enabling Technologies

Almost all OEMs—and many of their suppliers—have been working on creating a modern vehicle operating system (OS) for some time now. There is still no comprehensive and widely accepted definition of what comprises a vehicle OS. However, most definitions contain the following key components: specialized software runtimes for different functional domains, hardware and software decoupling via standardized APIs, and over-the-air (OTA) software updates.

When following this definition, an SDV is a key part of the vehicle OS—or maybe even synonymous with it. SDVs are enabled by a set of technologies that are efficient and continuous, even after the start of production (SOP) delivery of new, digital vehicle features, combining on-board and off-board components into an integrated end-to-end software architecture. We will start by looking at emerging electrical and electronic (E/E) architecture and how it can work with a service-oriented architecture (SOA). Key elements of E/E and SOA are hardware abstraction, vehicle APIs, and the SDV tech stack. Modern vehicles use OTA updates to support post-SOP updates, which will eventually be the foundation for vehicle app stores. Finally, we will look at how artificial intelligence (AI) can augment the software-centric vehicle.

Foundation: E/E Architecture

Today, so-called E/E architecture describes the overall design and layout of electrical and electronic systems in a vehicle. This architecture encompasses the distribution of power, data, and control signals throughout the vehicle as well as the integration and interconnection of various E/E components and systems. Many vehicles still implement a domain-centric E/E architecture where different vehicle domains, such as the powertrain, chassis, passenger compartment, and body, are logically grouped together and connected by dedicated bus systems, such as the Controller Area Network (CAN) bus. The CAN bus is a signal-based protocol designed to allow electronic control units (ECUs) and other compute nodes in a vehicle to communicate with each other in a reliable, priority-driven way.

Since the domain E/E architecture results in very complex and heavy vehicle-wiring harnesses, OEMs are using so-called zonal architectures, which aim to group different vehicle sensors and actuators according to their physical location in the vehicle. In a zonal E/E architecture, wiring harnesses become less redundant, allowing for simplified connections within individual zones, reducing complexity and weight, and enabling easier integration of new features and technologies. Zonal architectures usually combine dedicated zone controllers with high-performance vehicle computers. The zone controllers are locally connected to various sensors and actuators, often using different legacy/heritage bus systems, such as CAN, Local Interconnect Network (LIN), and FlexRay. The zone controllers are then connected with each other and with the high-performance vehicle computers via new on-board, high-speed networks based on Ethernet (the foundation of today's internet).

Vehicle APIs

The first step toward a service-oriented architecture for digital on- and off-board services is to provide a hardware abstraction via vehicle APIs. Today, developing new on-board features usually involves a complex and lengthy alignment process among many departments of the OEM. This is because all signals within a given on-board domain are communicated via a shared bus system (e.g., the CAN bus). For each vehicle type, a CAN matrix defines which ECUs send which message under which conditions and with which cycle time,

which ECUs receive which messages, and how the messages are structured and prioritized. This results in a tightly coupled architecture that requires very close alignment on technical and organizational levels. To get from here to a loosely coupled, service-oriented architecture, a new level of hardware abstraction is required.

From the perspective of the service consumer (i.e., user of digital applications), the vehicle should provide a set of well-defined APIs that provide an abstraction of the vehicle's functions. On the lowest level, these are the sensors and actuators of the vehicle. A good example of this open industry standard is the Vehicle Signal Specification (VSS) defined by the Connected Vehicle Systems Alliance (COVESA VSS). COVESA VSS defines a tree-like API structure for accessing vehicle sensors and actuators as signals (see Figure 3-1).

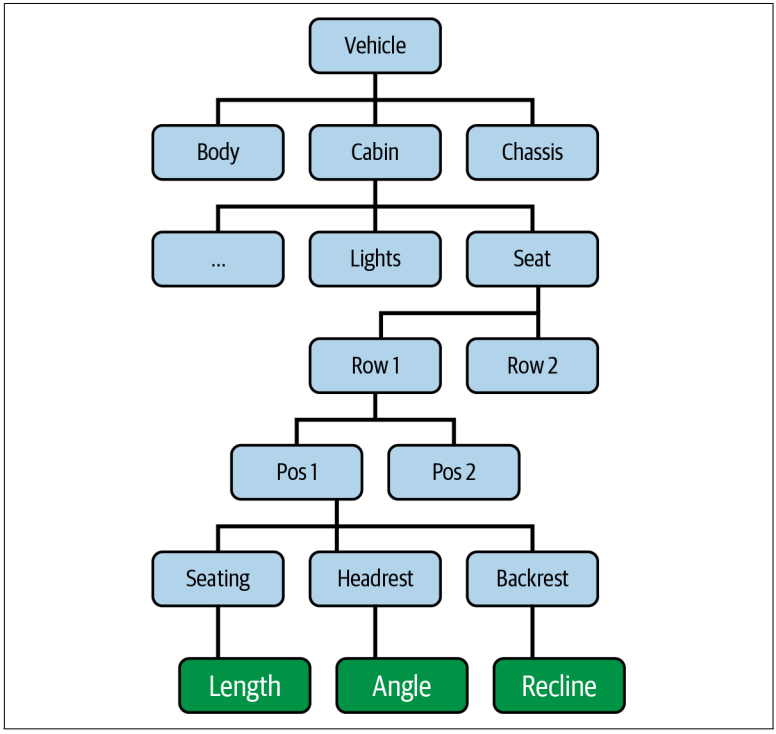

Figure 3-1. Vehicle signal API tree (based on COVESA VS)

For example, `Vehicle.Cabin.Seat.Row1.Pos1.Headrest.Angle` would give an application developer access to the actuator that controls the angle of the headrest of the front left seat. This is a level of abstraction suitable for developers used to cloud-native or smartphone development. Of course, these relatively low-level signal-to-service APIs have to be augmented with higher-level orchestration services over time.

However, there are some issues with this approach. First, as discussed before, it can be challenging to map such a high-level software API to the underlying, complex E/E architecture. Second, there is the question of how to ensure the functional safety of APIs that may impact vehicle physics. And third, it is necessary to solve for Quality of Service (QoS) aspects of such an API (e.g., real-time requirements). We will look at all of these aspects in the following sections.

Vehicle SOA: Encapsulating the E/E Architecture with Vehicle APIs

The issue with the E/E architecture perspective is that it is modeled after the physical design of the vehicle, including actuators, sensors, on-board networks, and compute nodes. A key prerequisite for SDVs will be to encapsulate the hardware-focused E/E architectures with a vehicle service-oriented architecture (SOA), as depicted in Figure 3-2.

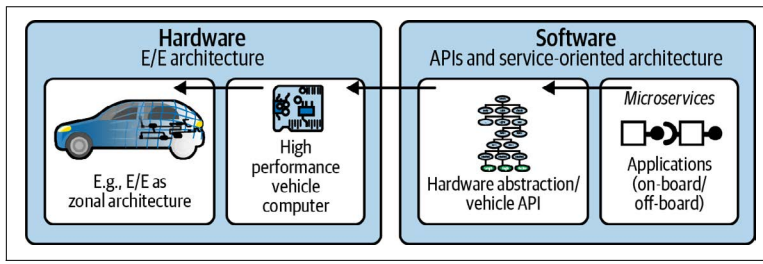

Figure 3-2. SOA encapsulating the E/E architecture

On the E/E hardware side, high-performance computers run the SDV functions. These computers are physically connected with the other on-board components, including smaller compute nodes, sensors, and actuators. The translation from the hardware side to the software side can happen on these high-performance com-

puters; hardware functions are exposed via APIs. These APIs are the foundation for the communication between application-level microservices and the underlying hardware. A *microservice* is an encapsulated piece of software that implements a specific function. Interaction between microservices always happens via APIs. These can either be APIs abstracting a specific hardware function or application-level APIs. An *application* is a collection of microservices that are orchestrated to provide a specific feature or digital service.

Layers of the Vehicle SOA

SOAs introduce a level of architectural layering that helps deal with the individual characteristics of the different microservices involved. In the world of the internet, for example, the frontend layer of an application would include microservices that change frequently due to continuous optimization of the user interface while the basic services layer would include more stable, data-centric services that change much less frequently. This type of layering is essential for the efficient evolution of complex systems.

In the automotive world and SDV, this layering is different and has multiple dimensions. The first dimension we need to look at is on-board versus off-board. The second dimension is defined by the functional safety and real-time requirements of the contained microservices and the vehicle event chains encapsulated by those microservices.

Putting these dimensions together with what we discussed earlier about E/E architectures and vehicle APIs, we get a service-oriented architecture for the SDV as described in Figure 3-3. At the top, the green layer indicates the environment for QM-only microservices hosted in the cloud. A set of vehicle-to-cloud APIs connects on-board services with off-board services. In the middle part, the SDV layer includes both QM-only and ASIL A/B microservices, which are hosted in different environments. Further south in this architecture, the physical vehicle functions can be found, made accessible through a signal-to-service API (e.g., based on COVESA VSS). The signal-to-service API must encapsulate the mapping of the APIs to the E/E architecture of the vehicle. For example, the API must know which zone controller would actually host the functions required to support `Vehicle.Cabin.Seat.Row1.Pos1.Headrest.Angle` (see

Figure 3-1). The zone controller (in this case the "Zone FL" of the front left) would, in turn, need to provide a software proxy that can translate the API onto the matching CAN signal so that the angle of the front left headrest is changed.

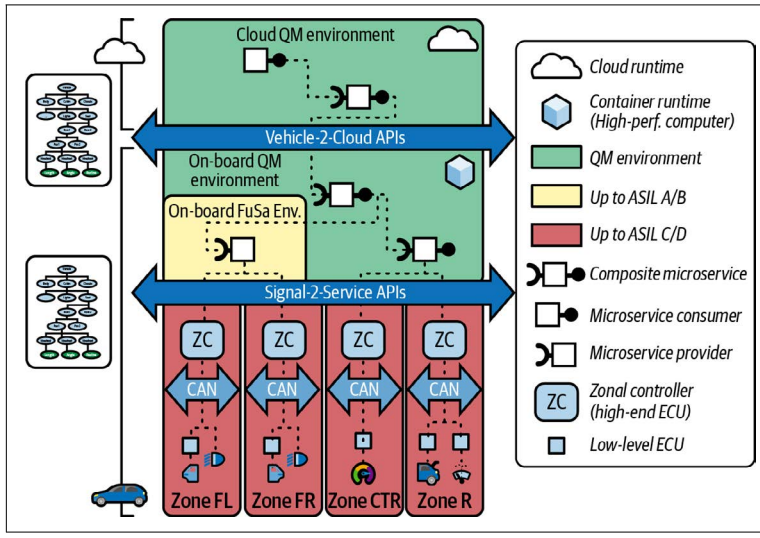

Figure 3-3. Service-oriented architecture for the software-defined vehicle

Ensuring Functional Safety in the Vehicle SOA

A key benefit of the multilayered vehicle service architecture is that we now have several options for functional safety. Let's look at a concrete example in Figure 3-4, a function in a smartphone app that remotely opens the trunk of a vehicle.

The problem with the "open trunk" function is that it can only be safely executed if the vehicle is in a safe state (i.e., not moving). So, the question is: who will perform this check, and how? The caller itself cannot perform the check—this would violate the loose coupling principles of the service-oriented architecture, as the service implementation cannot assume clients always properly follow a certain protocol. This is certainly true for clients calling from the cloud, but also for clients calling from a QM environment on board the vehicle. Indeed, this is the whole point of the QM environment; it provides none of the ASIL QoS properties. This means that the on-board "open trunk" service has to perform the required safety

checks itself. Assuming that the service interface is exposed to QM clients, there are now at least two options.

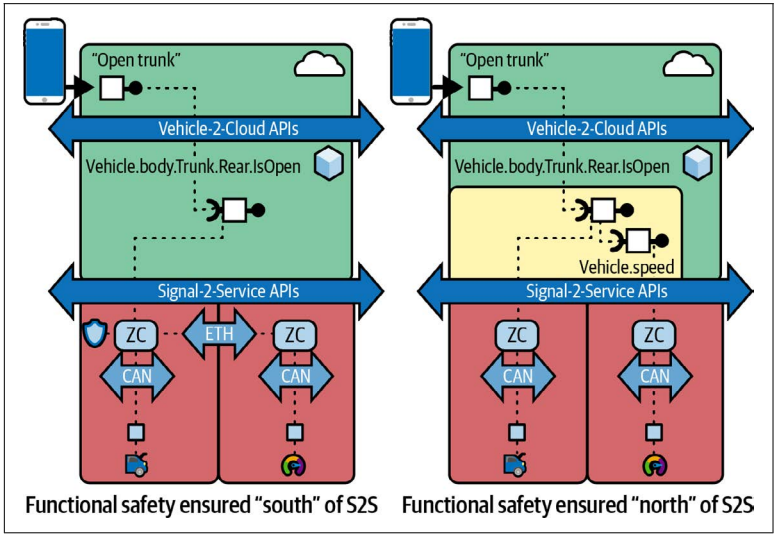

Figure 3-4. Vehicle SOA and Functional Safety–two examples

In the first option (Figure 3-4, left), the on-board "open trunk" service (`Vehicle.Body.Trunk.Rear.IsOpen`) runs in a QM environment and is therefore not considered safe. It might provide some additional services, but eventually the safety check must be done from within the ASIL safety environment. In our case, this would be the zone controller for the trunk. Here, the system must check the current vehicle speed before opening the trunk. However, in this example, a different zone controller manages the speed signal (i.e., the zone controller for the powertrain). So the zone controller that manages the trunk must access the zone controller that manages the speed signal before communicating with the low-level ECU controlling the trunk. And all of this must happen in real time (which is why this check is performed on the zone controller, not in the QM layer above). This does not necessarily mean it has to happen in a fraction of a millisecond, but it has to happen within a guaranteed time interval. For this to happen, the two zone controllers must be in more direct communication, such as a direct link supporting so-called Time-Sensitive Networking (TSN) via high-speed Ethernet.

The second option (Figure 3-4, right) assumes that the on-board "open trunk" service is executed in a safety environment matching the ASIL levels required for this function. This means the exposed interface can still be called from a QM client (e.g., an event sequence originating in the cloud), but the implementation behind the interface is safe. If there is another safety-compliant `Vehicle.Speed` service (and a safe way for the trunk service to call the speed service), then this can now all be done in the same environment. The benefit is that the speed service is provided as a real microservice that can be reused by QM as well as ASIL services. However, this requires a service architecture capable of orchestrating microservices with ASIL QoS levels.

Both options presented here are valid. The first option makes fewer demands on the integration levels within the environment and might be easier to implement with current technologies. The second option has more potential to enable advanced cross-domain use cases, but requires a more advanced ASIL-compliant runtime environment. The following discussion will look at the SDV tech stack required to support both options.

SDV Tech Stack for the Vehicle SOA

To understand how a vehicle SOA can support architectural layers with different QoS levels, we need to add another dimension—adding the hardware, operating system, and middleware/application environment. This is shown in Figure 3-5.

The cloud layer shown here is as to be expected. The hardware includes generic central processing units (CPUs) and graphics processing units (GPUs), which are used today for processing machine learning workloads in the cloud. On the operating systems (OS) level, we usually find a general-purpose OS like Linux. Hypervisors or virtual machines are used for running multiple OS instances on shared hardware, which is important for scalability and enabling cloud elasticity. Modern clouds also provide very rich middleware and application runtimes.

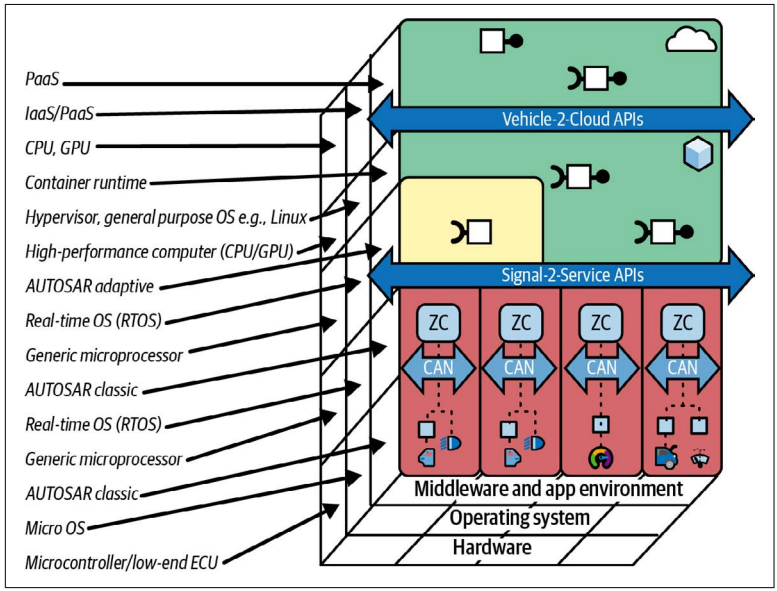

Figure 3-5. The SDV tech stack

Next we have the on-board environment for QM-only microservices and applications. This type of environment aims to replicate as much as possible the rich tech stack found today in the cloud while adding some vehicle-specific aspects. These will include, for example, faster start-up times, energy efficiency, and support for managing highly distributed instances that are not always online (e.g., large vehicle fleets, as opposed to central cloud data centers). The OS in this layer will usually be a general-purpose OS combined with hypervisors for virtualization. This will run on a generic high-performance vehicle computer, potentially with additional GPUs for AI/ML workloads.

The next layer has to support high QoS levels and consequently requires a more traditional high-availability environment, usually with real-time support. A widely used OS in this area is QNX from BlackBerry. The zone controllers will probably share a similar setup. A real-time-capable middleware will be required to link microservices from the environment north of the S2S API with those residing south of it (e.g., on zone controllers).

Finally, the zone controllers connect to lower-level ECUs. Especially for the control of specific vehicle sensors and actuators, microcontrollers are used. These are integrated circuits (ICs), usually repre-

senting a complete system on chip (SoC), including the processor core, memory, and IO, all inside one discrete package. Specialized environments for low-level but highly efficient and very targeted embedded functions are used here. Between the zone controller and the different low-level ECUs and microcontrollers, different bus systems have to be supported, including CAN, LIN, and FlexRay.

OTA: Over-the-Air Updates

In the past, the automotive industry aimed to freeze the hardware and supporting software of a new vehicle generation at the SOP. Post-SOP changes to the supporting software were usually made only if serious problems needed to be fixed.

Of the 70 to 100 ECUs and controllers usually found on a traditional vehicle, the majority would never be updated after the initial software flashing as part of the manufacturing process. If updates were needed, a dedicated update campaign would be run for each affected variant ("push"). Updates are usually manually packaged based on extensive variant research and validation.

In the future, many OEMs envision a rich application ecosystem developing for their vehicles. Software updates will no longer be done only for quality reasons but can happen on demand. Customers will select which new applications and features they want to download and activate ("pull"). These new applications can span different domains, ranging from infotainment and well-being to cabin comfort and driving performance. However, this increasing individualization will also lead to a dramatic rise in variants of hardware/software combinations. The next generation of OTA will have to address this (Figure 3-6).

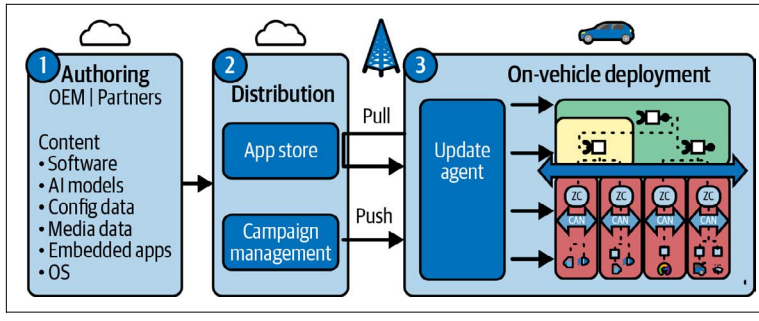

Figure 3-6. Modern over-the-air update architecture

The Vehicle App Store

The holy grail of the SDV is the vehicle app store. The smartphone industry has proven the huge potential of applications and content generated by partners and external developers, made accessible to customers on demand and post-SOP. Now OEMs are jumping on the bandwagon. Replicating the success story of smartphone app stores in the automotive industry requires several things. Figure 3-7 gives an overview of the technical prerequisites.

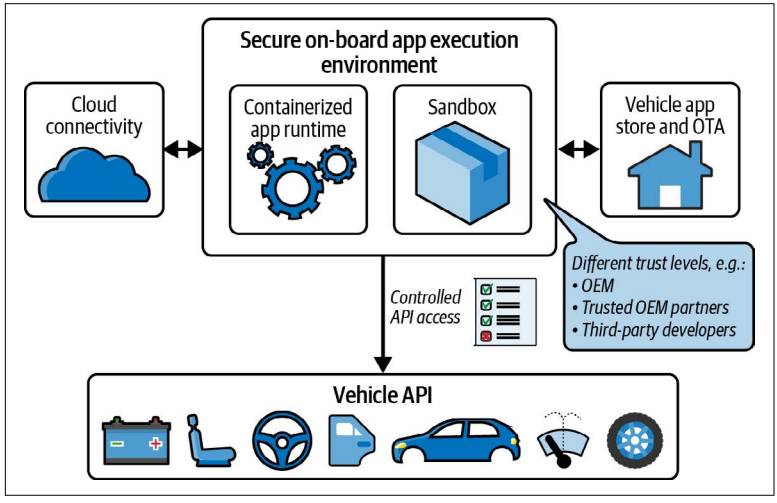

Figure 3-7. Vehicle app store

First, vehicles need a secure on-board application runtime environment for the execution of applications downloaded via the vehicle app store. This runtime is often referred to as a sandbox. It is important that this sandbox be secure, not allowing code to break out and access other parts of the vehicle. A virus that takes control of a vehicle can have deadly consequences for the vehicle's occupants or bystanders.

Second, the system architecture must ensure that applications running inside the sandbox can only interact with other parts of the vehicle via a set of controlled APIs. Most likely, OEMs will have to define different trust levels for their APIs, differentiating between API access through OEM-developed applications, trusted partner apps, and potentially apps developed by completely unknown third parties. Already, some vehicles available today provide app stores

for in-vehicle infotainment. In the future, vehicle app stores should also provide access to sensor APIs to unleash the creativity of the global developer community. They may even offer safe access, so selected actuators might be an option (e.g., in the body and comfort domain).

While the smartphone juggernauts were already able to position apps via Apple Car Play and Android Automotive, many OEMs are now starting to offer app stores for applications running natively in the vehicle. This journey starts in the infotainment area, introducing apps designed to run natively on the built-in infotainment system screen. Examples include apps for music and podcasting, video conferencing, weather, gaming, news, parking, and electric vehicle (EV) charging. It remains to be seen how well these vehicle infotainment applications will be able to compete with their smartphone counterparts.

To be better differentiated, the next generation of in-vehicle apps is likely to make use of vehicle-specific features. Once apps are able to interact in a meaningful and safe way with the vehicle, a new experience can be created. Being able to access data from vehicle sensors—and potentially even control some of the vehicle actuators—will create a new type of application. For example, being able to access in-vehicle sensors, such as cameras, radar, and microphones, could create a new generation of health and well-being apps. This would help OEMs come closer to the vision of a "habitat on wheels."

SDV and AI

While AI has already become a hot topic, the release of ChatGTP has moved the discussion from "software will eat the world" to "AI will eat the world." AI is disruptive on many levels, and the automotive world is no exception. So, do we need an "AI-defined vehicle" instead of (or in addition) to a software-defined one? Where in the SDV architecture does AI play a role? The answer is: potentially in all the layers of the SDV architecture (see Figure 3-8).

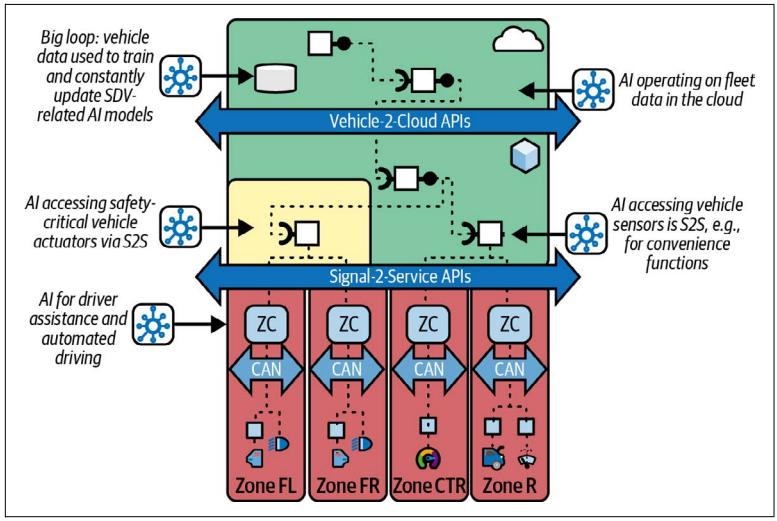

Figure 3-8. SDV and AI

On the smartphone, AI can be used to control access to the vehicle or specific vehicle functions (e.g., via face recognition). In the cloud, AI can operate on vehicle fleet data (e.g., to support vehicle routing, optimize the EV charging experience, perform traffic analysis, improve battery performance, etc.).

On-board, we are seeing different types of AI. For highly automated driving, AI is deeply integrated with the vehicle's E/E architecture, providing object identification (e.g., recognizing a child on a bicycle in front of the car) and controlling the vehicle's trajectory (e.g., braking for the aforementioned child). In addition, there seems to be a huge potential for AI-enabled long-tail applications. These applications require much less investment for their development and will probably have a lesser overall impact individually but still have a huge impact overall. The smartphone app stores have proven that if the boundaries of entry are sufficiently low, a plethora of new, creative applications will be created. This should also be true for AI-enabled vehicle apps.

Combining AI with data from in-vehicle sensors alone could be a game changer (e.g., for infotainment and well-being). If this was then combined with SDV-enabled functions, a whole new generation of in-vehicle applications could be created (e.g., cabin comfort applications that use sensor data to change the vehicle's ambience by accessing in-cabin light, HVAC, seat massage functions, and so

forth). The use of AI will reach far beyond vehicle apps. From voice assistance to predictive maintenance, from fleet operations to automated driving to AI-assisted development toolchains, AI will be a game changer for the way we design, develop, operate, and interact with vehicles.

CHAPTER 4

Value Stream Management for the SDV

The internet is built on sophisticated, constantly evolving technologies. Consequently, many successful internet businesses are built on a technology culture. However, this reliance on technology can sometimes make it difficult to focus on building customer-centric products, increasing competitiveness and generating revenue. Therefore, many companies have adopted Value Stream Management (VSM) as a best practice for their digital businesses. VSM is a set of practices designed to ensure that new digital features are delivered fast and efficiently and that they deliver clear customer value.

So how can VSM be applied to the software-defined vehicle? Let's have a look.

Working at Different Speeds

What is maybe the most important realization in the context of the SDV is that there cannot be a single value stream. Our discussion of the "clash of two worlds" and the following technical discussions have shown that different requirements need different, specialized approaches. The digital vehicle experience must be delivered by a digital value stream, which adheres to Agile best practices, while the physical vehicle experience must be delivered by a value stream that adheres to the rigorous and "first time right" thinking required